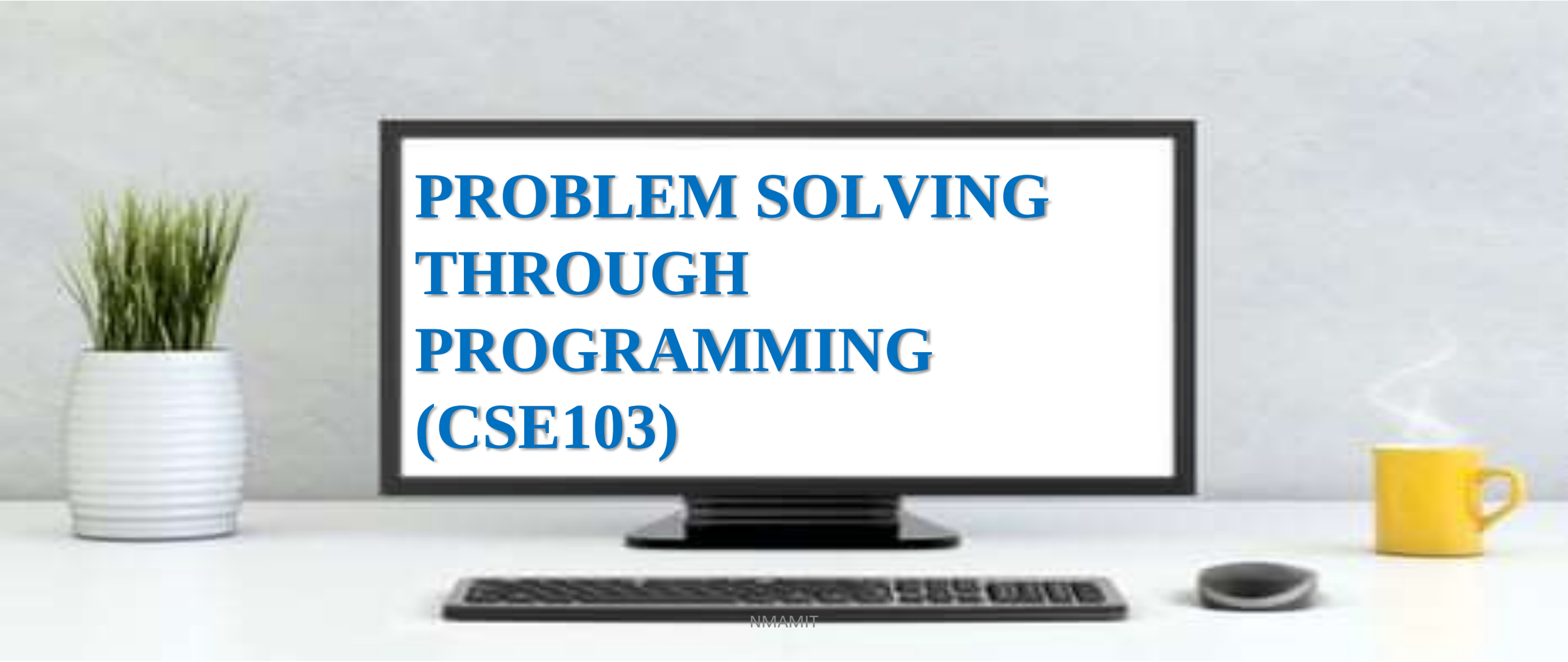
NITTE
(Deemed to be University)

NMAM INSTITUTE OF TECHNOLOGY



PROBLEM SOLVING THROUGH PROGRAMMING (CSE103)

UNIT-5
Structures and Files

By,
Prithviraj Jain
CSE, NMAMIT

Syllabus:

Structures: Defining Structure, Declaration of Structure Variable, Accessing Structure members, copying and comparing structure variable, operation on individual member, nesting of structures, Array of structures. Application of pointers and function on Structures.

Files: Basic file operations: Open, Close, Read, Write

Structures

Definition:

- **Structure** is a user-defined datatype which allows us to combine data of different types together.
- **Structure** helps to construct a complex data type which is more meaningful. It is similar to an Array, but an array holds data of similar type only.
- In **structure**, data is stored in the form of records.
- Structure defines a new data type which is a collection of primary and derived data types.

Syntax for defining a Structure

struct [structure_name]

{

  //member variable 1

  //member variable 2

  //member variable 3

  ...

}**[structure_variables];**

- struct is a keyword.
- tagname specifies the name of the structure but tagname is optional.
- member1, member2 are the data items of different types

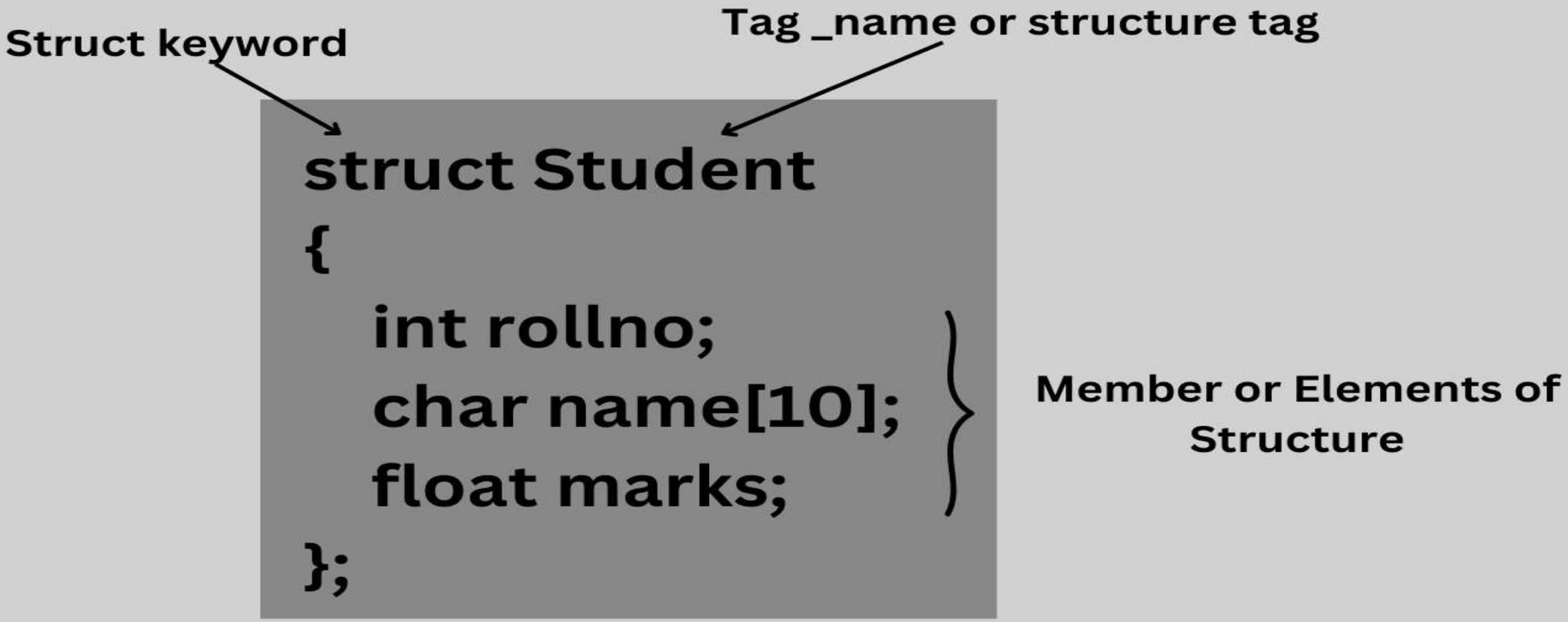
Example:1

•To store the data of student like student name, age, address, id etc. using structures:

```
struct Student
{
   char name[25];
   int age;
   char branch[10];
   char gender; // F for female and M for male
};
```

- Here, **struct Student** declares a structure to hold the details of a student which consists of 4 data fields namely name, age, branch and gender. These fields are called **structure elements or members**.
- Each member can have different type of data
- **Student** is the name of the structure and is called as the **structure tag**.

Structure in C



Declaring Structure Variables

Structure variables can be declared in two ways:

1) Declaring Structure variables separately

```
struct Student
{
  char name[25];int age;  char branch[10];
  char gender; //F for female and M for male
};
```

struct Student S1, S2; //declaring variables of structure Student

2) Declaring Structure variables with structure definition

```
struct Student
{
  char       name[25];  int age;
  char branch[10];
  char gender; //F for female and M for male
}S1, S2;
```

- Here, S1and S2 are variables of structure Student.

Accessing Structure Members

- Structure members can be initialized and accessed in a number of ways.
- Structure members have no meaning individually without the structure.
- In order to assign a value to any structure member, the member name must be linked with the structure variable using a **dot .** operator
- Dot operator is also called as period or member access operator.
- Structure members cannot be initialized with declaration.

How to initialize structure members?

Structure members **cannot be** initialized with declaration.

For example the following C program fails in compilation.

```
Struct student
{
int age=20; // COMPILER ERROR: cannot initialize members here char name[20]=“prithvi”; // COMPILER ERROR: cannot initialize members here
};
```

```
struct student
{
int age;
char name[20];
};
int main()
{
struct student s;
s.age=20;
strcpy(s.name,”prithvi”);
}
```

```
struct student
{
int age;
char name[20];
};
int main()
{
struct student s={25,”ramesh”};
return 0;
}
```

- When a member variable is declared, no memory is allocated for it. Memory is allocated only when variables are created.
- Structure members can be initialized using curly braces '{}'.

For example, following is a valid initialization.

```
struct Point
{
   int x, y;
};
int main( )
{
   // A valid initialization - member x gets value 0 and y gets value 1.
   // The order of declaration is followed.
   struct Point p1 = {0, 1};
}
```

Example for accessing the structure elements

Structure members or elements are accessed using **dot (.)** operator.

```
#include<stdio.h>
struct Point
{
   int x, y;
};
int main()
{
   struct Point p1 = {0, 1};
   // Accessing members of point p1
   p1.x = 20;
   printf ("x = %d, y = %d", p1.x, p1.y);
   return 0;
}
```

Output :
x = 20, y = 1

Example:2

```
#include<stdio.h>
#include<string.h>
struct Student
{
    char name[25];
    int age;
    char branch[10];
    char gender;
};
int main()
{
    struct Student s1;
    /*s1 is a variable of Student type and
    age is a member of Student*/
    s1.age = 18;
    strcpy(s1.name, "Viraaj");
    printf("Name of Student 1: %s\n",
    s1.name);
    printf("Age of Student 1: %d\n",
    s1.age);
    return 0;
}
```

Copying and Comparing Structure Variables

Two variables of the same structure type can be copied the same way as ordinary variables. If persona1 and person2 belong to the same structure, then the following statements are valid.

person1 = person2;

person2 = person1;

C does not permit any logical operators on structure variables. In case, we need to compare them, we may do so by comparing members individually.

person1 = = person2 (Equality) //Invalid statement

person1 ! = person2 (Not Equal) //Invalid statement

```
struct class
{
int num;
char name [20];
float marks;
};
```

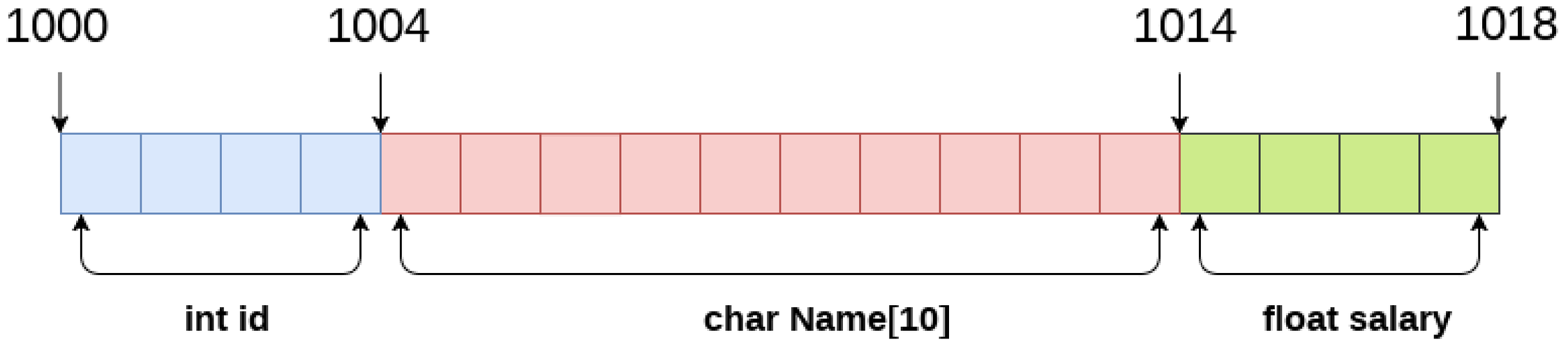
Not permitted

```
if (s1==s2)
{
……….
}
```

```
int main ( )
{
int x;
struct class s1 = {111, “Rao”, 72.50};
struct class s2;
s2 = s1;   //legal
if (s2.num==s1.num)
printf(“both numbers are equal\n”);
else
printf(“both numbers are not equal\n”);
return 0;
}
```

Size of a structure:

```
struct employee
{
    int id;
    char name[10];
    float salary;
};
```




```
struct Employee
{
     int id;
     char Name[10];
     float salary;
} emp;
```

sizeof (emp) = 4 + 10 + 4 = 18 bytes

where;
sizeof (int) = 4 byte
sizeof (char) = 1 byte
sizeof (float) = 4 byte

Operation on Individual Members of Structures

- Once you create a structure, you can access and operate on its **individual members** using the **dot (.) operator** for structure variables and the **arrow (->) operator** for pointers to structures.

```
#include <stdio.h>
struct Student {
    int rollNo;
    float marks;
    char grade;
};
```

```
int main() {
    struct Student s1;  // structure variable
    // Assigning values to individual members
    s1.rollNo = 101;
    s1.marks = 87.5;
    s1.grade = 'A';
    // Accessing and printing members
    printf("Roll No: %d\n", s1.rollNo);
    printf("Marks  : %.2f\n", s1.marks);
    printf("Grade  : %c\n", s1.grade);
    return 0;
}
```

```
int main() {
    struct Student s2 = {102, 92.0, 'A'};
    struct Student *ptr = &s2; // pointer to structure
    // Accessing members using arrow operator
    printf("Roll No: %d\n", ptr->rollNo);
    printf("Marks  : %.2f\n", ptr->marks);
    printf("Grade  : %c\n", ptr->grade);
    return 0;
}
```

Operation on Individual Members of Structures

Example program illustrating operations on structure members:

```
#include<stdio.h>
#include<string.h>
struct student
{
        int rno;
        char name[10];
        int marks, age;
};
int main()
{
        int m;
        //structure variable initalization
        struct student 1={2,"Geetha",89,18};
        struct student s2;
        s2=s1;
        printf("\nDetails of student 1:\n");
        printf("\n roll number: %d",s1.rno);
        printf("\n name :%s",s1.name);
        printf("\n marks: %d",s1.marks);
        printf("\n age: %d",s1.age);
        printf("\n\n");
        printf("Details of student 2:\n");
        printf("\n roll number: %d",s2.rno);
        printf("\n name :%s",s2.name);
        printf("\n marks: %d",s2.marks);
        printf("\n age: %d",s2.age);
        //comparsion of two student details
        m=((s1.rno==s2.rno)&&(s1.marks==s2.marks))?1:0;
        if(m= =1)
                printf("\n Both the students are same");
        else
                printf("\n Both the students are not same");
}
```

Nested Structures

- A **nested structure** in C means defining a structure **inside another structure**. This is useful when a "whole" object contains another "sub-object" with its own members.
- two ways:
  - **Directly inside** the parent structure.
  - **Separately** and then include as a member.

Directly inside the parent structure.

```
#include <stdio.h>
struct Student {
    int rollNo;
    char name[30];
    struct Marks {
// Nested structure
        int maths;
        int science;
        int english;
    } m;
// variable of nested structure
};
int main() {
    struct Student s1;
    // Assigning values
    s1.rollNo = 101;
    strcpy(s1.name, "Anita");
    // copy string into name
    s1.m.maths = 90;
    s1.m.science = 85;
    s1.m.english = 88;
    // Display
    printf("Roll No: %d\n", s1.rollNo);
    printf("Name   : %s\n", s1.name);
    printf("Marks  : Maths=%d, Science=%d,
English=%d\n", s1.m.maths, s1.m.science,
s1.m.english);
    return 0;
```

Separately and then include as a member.

```
#include <stdio.h>
struct Marks {
    int maths;
    int science;
    int english;
};
struct Student {
    int rollNo;
    char name[30];
    struct Marks marks;
// Nested as a member
};
```

```
int main() {
    struct Student s1 = {101, "Anita", {90, 85, 88}};
    printf("Roll No: %d\n", s1.rollNo);
    printf("Name   : %s\n", s1.name);
    printf("Maths  : %d\n", s1.marks.maths);
    printf("Science: %d\n", s1.marks.science);
    printf("English: %d\n", s1.marks.english);
    return 0;
}
```

Nested Structures

- We can create structures within a structure in C programming

```
int main() {
  // initialize complex variables
  num1.comp.imag = 11;
  num1.comp.real = 5.25;
  // initialize number variable
  num1.integer = 6;
  // print struct variables
  printf("Imaginary Part: %d\n",
  num1.comp.imag);
  printf("Real Part: %.2f\n", num1.comp.real);
  printf("Integer: %d", num1.integer);
  return 0;
}
```

Suppose, to set **imag** of **num2** variable to **11, we can do it:** **num2.comp.imag = 11;**

Passing Structures to Functions

- **Pass by value**

```
#include <stdio.h>
struct Student {
    int rollNo;
    float marks;
};
// Function accepts structure as value
void display(struct Student s) {
    printf("Roll No: %d, Marks: %.2f\n", s.rollNo, s.marks);
}
int main() {
    struct Student s1 = {101, 88.5};
    display(s1);  // copy passed
    return 0;
}
```

- **Pass by pointer**

```
#include <stdio.h>
struct Student {
    int rollNo;
    float marks;
};
// Function accepts pointer to structure
void updateMarks(struct Student *s, float newMarks) {
    s->marks = newMarks;  // use -> for pointer access
}
int main() {
    struct Student s1 = {101, 88.5};
    updateMarks(&s1, 92.0);  // pass address
    printf("Updated Marks: %.2f\n", s1.marks);
    return 0;
}
```

Using Pointers with Structures

- **Dot (.)** operator → for normal structure variable.
- **Arrow (->)** operator → for pointer to structure.

- struct Student s = {101, 80.0};
- struct Student *ptr = &s;

- printf("%d", ptr->rollNo); // arrow
- printf("%d", (*ptr).rollNo); // equivalent, but longer

File Handling in C++

01 Opening a File

02 Reading File

03 Writing to a File

04 Closing a File

File handling

A **file** represents a sequence of bytes on the disk where a group of related data is stored. File is created for permanent storage of data.

In C language, we use a structure **pointer of file type** to declare a file.

Syntax: FILE ***file_pointer**;

Example: FILE ***fp**;

Introduction to File Operations

- Files store data permanently on storage devices.
- File operations allow programs to access and manipulate files.
- Common file operations include Open, Close, Read, and Write.
- Files can be text files or binary files.
- Proper file handling ensures data integrity and efficient resource use.

Introduction to File Operations

FILE:A file is a container in computer storage devices used for storing data, information, settings, or commands used with a computer program.

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- A file represents a sequence of bytes on the disk where a group of related data is stored.
- File is created for permanent storage of data.
- When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and the program.
- In C language, we use a structure pointer of file type to declare a file.

```
FILE *fp;
```

Major Operations on File

- Create a file
- Open a file
- Input/output(accessing the content) needs file pointer
- Close a file

functions used in C to perform basic file operations

- fopen()
- fclose()
- fprintf()
- fscanf()
- fputc()
- fgetc()
- fputs()
- fgets()
- feof()

Opening a File

- Opening a file prepares it for reading or writing.
- Use functions like fopen() in C.
- File modes include read (r), write (w), append (a), and read/write (r+).
- Opening a non-existent file in write mode creates a new file.
- Always check if the file opened successfully (e.g., check for NULL).

OPENING A FILE OR CREATING A FILE:

- The fopen() function is used to create a new file or to open an existing file.
- We must open a file before it can be read, write, or update
- General Syntax :
- ***fp = FILE *fopen(const char *_filename_, const char *_mode_);**

Opening a File

Mode	Description
r	opens a text file in reading mode **fp = fopen(“input.*txt*", “*r*");**
w	opens or create a text file in writing mode. **fp = fopen(“*input.txt*", “*w*");**
a	opens a text file in append mode.
r+	opens a text file in both reading and writing mode.
w+	opens a text file in both reading and writing mode.
a+	Open for reading and appending (writing at end of file).

Opening a File

Mode	Description
rb	opens a binary file in reading mode.
wb	opens or create a binary file in writing mode.
ab	opens a binary file in append mode.
rb+	opens a binary file in both reading and writing mode.
wb+	opens a binary file in both reading and writing mode.
ab+	opens a binary file in both reading and writing mode.

Writing to a File

- Writing saves data from the program to a file.
- Functions: fputc(), fputs(), fwrite() in C; write() in Python.
- Writing in write mode overwrites the file content.
- Append mode adds data at the end without deleting existing content.
- Always ensure the file is opened in write or append mode before writing.

Writing to a File

fprintf():

- fprintf() function is used to write formatted data into a file.

Declaration:

int fprintf(FILE *fp, const char *format, …);

In a C program, we use fprinf() as below.

fprintf (fp, “%s %d”, “var1”, var2);

Where, fp is file pointer to the data type “FILE”.

- var1 – string variable
- var2 – Integer variable

Reading from a File

- Reading retrieves data from a file to a program.
- Functions: fgetc(), fgets(), fread() in C; read(), readline() in Python.
- Can read data character-wise, line-wise, or in blocks.
- End-of-file (EOF) signals no more data to read.
- Handle errors like trying to read from a file not opened for reading.

Reading from a File

fscanf() function:

fscanf() function is used to read formatted data from a file.

Declaration:

```
int fscanf(FILE *fp, const char *format, …) ;
```

In a C program, we use fscanf() as below.

```
fscanf (fp, “%d”, &age);
```

Where, fp is file pointer to the data type “FILE”.

Age – Integer variable

This is for example only. You can use any specifiers with any data type as we use in normal scanf() function.

CLOSING A FILE

The fclose() function is used to close an already opened file.

General Syntax :

int fclose(FILE **fp*);

Closing a File

- Closing a file releases system resources.
- Use fclose() in C or close() in Python.
- Always close files after operations to avoid memory leaks or corruption.
- Closing flushes any buffered data to the disk.
- Trying to access a file after closing it results in an error.

Best Practices and Errors in File Handling

- Always check return values of file operations for errors.
- Handle exceptions or errors gracefully to avoid crashes.
- Use binary mode (rb, wb) for non-text files.
- Avoid reading or writing beyond the file boundaries.
- Close all files properly even if an error occurs during processing.

END OF UNIT 5